```
import pandas as pd
import ta
import yfinance as yf
from datetime import datetime
import os
import argparse
import sys

# ================================================================================
# 📝 STRATEGY DOCUMENTATION & TUNING GUIDE FOR USERS
# ================================================================================
"""
MOMENTUM SCANNER ENGINE CONFIGURATION MANUAL
--------------------------------------------------------------------------------
Modify the parameters below to adapt the scanner to your risk tolerance.

1. TREND BASICS (Lengths):
   - EMA_FAST_PERIOD (50) & EMA_SLOW_PERIOD (200): Controls the structural trend.
     Do not drop EMA_SLOW_PERIOD below 100, or it will conflict with multi-day
swing logic.

2. SPEED PARAMETERS (Oscillators):
   - CFG_ADX_TREND_STRONG (Default: 25.0 | Range: 20.0 to 30.0):
     Higher value means you only scan stocks moving like rockets. Lower values
allow slower trends.
   - CFG_ADX_EXTREME (Default: 50.0 | Range: 45.0 to 60.0):
     Triggers a 'DO NOT BUY' protection. Trends above 50 are rare, vertical, and
unsustainable.
   - CFG_RSI_OVERBOUGHT (Default: 70.0 | Range: 65.0 to 85.0):
     Defines the price ceiling. Set to 80-85 if you want to allow hyper-aggressive
momentum chasing.
   - CFG_STOCH_OVERSOLD (Default: 20.0 | Range: 15.0 to 35.0):
     Defines the depth of a pullback. Setting to 30 catches milder, shallower dips
in strong trends.

3. VOLATILITY PROTECTION (Rubber Band / Distance Filters):
   - CFG_ATR_SAFE_MULT (Default: 1.5 | Range: 1.0 to 1.8):
     Maximum distance from EMA50 for a safe entry. Keeps you buying near the
support floor.
   - CFG_ATR_EXT_MULT (Default: 2.5 | Range: 2.2 to 3.5):
     The ultimate risk barrier. If price flies higher than '2.5 * ATR' away from
its EMA50,
     it is mathematically overextended. Increase to 3.0+ only if the entire market
is in a parabolic run.
--------------------------------------------------------------------------------
"""

# ================================================================================
# 1. PARAMETER VALUE ASSIGNMENTS
# ================================================================================
```

```
# Technical Indicator Lengths
EMA_FAST_PERIOD = 50
EMA_SLOW_PERIOD = 200
ADX_PERIOD      = 14
RSI_PERIOD      = 14
ATR_PERIOD      = 14
STOCH_K_PERIOD  = 9
STOCH_D_PERIOD  = 6

# Strategy Thresholds
CFG_ADX_TREND_STRONG = 25.0
CFG_ADX_EXTREME      = 50.0
CFG_RSI_OVERBOUGHT   = 85.0     # was 70.0

# 1. Allow trades on shallower, minor pullbacks (Increases hits)
#CFG_STOCH_OVERSOLD   = 20.0
CFG_STOCH_OVERSOLD = 35.0       # Was 20.0

CFG_STOCH_OVERBOUGHT  = 80.0

# ATR Multipliers for Distance from EMA50
CFG_ATR_SAFE_MULT    = 2.0        # was 1.5
CFG_ATR_EXT_MULT     = 3.5        # was 2.5

# Default Directories
DEFAULT_INPUT_CSV  = "d:/Tools/07_LiveScanner/watchlist.csv"
DEFAULT_OUTPUT_CSV = "d:/Tools/07_LiveScanner/scanner_historical_log.csv"
DEFAULT_FALLBACK_WATCHLIST = ["AAPL", "NVDA", "TSLA", "MSFT", "AMD", "AMZN",
"PARR"]

# ============================================================================
# 2. STATUS CODES & OUTPUT MESSAGES
# ============================================================================
STATUS_BUY_SIGNAL   = "BUY SIGNAL"
STATUS_WATCHLIST    = "WATCHLIST"
STATUS_HOLD         = "HOLD"
STATUS_DO_NOT_BUY   = "DO NOT BUY"
STATUS_IGNORE       = "IGNORE"
STATUS_ERROR        = "ERROR/SKIPPED"

# Strict sorting precedence array to process terminal block presentation order
SORT_ORDER_PRECEDENCE = [STATUS_BUY_SIGNAL, STATUS_WATCHLIST, STATUS_HOLD,
STATUS_DO_NOT_BUY, STATUS_IGNORE, STATUS_ERROR]

# Base message configurations
MSG_NO_TREND       = "No macro uptrend. Price below EMA50 or EMA50 below EMA200."
MSG_EXTREME_TOP    = "Hyper-extended top!"
MSG_WAIT_PULLBACK  = "Strong trend, but slightly extended. Wait for a pullback to
EMA50."
MSG_BUY_TRIGGER    = "Macro trend aligned, pullback completed, Stochastic crossed
```

```
up!"
MSG_HOLD_TREND     = "In a healthy upward phase. No new entries, but no exit
triggers yet."

# ============================================================================
# 3. CORE STRATEGY PIPELINE FUNCTION
# ============================================================================
def evaluate_stock_momentum(ticker_symbol: str) -> dict:
    result_template = {
        "ticker": ticker_symbol, "status": STATUS_ERROR, "message": "",
        "price": None, "ema_50": None, "ema_200": None, "macd": None,
        "macd_signal": None, "adx": None, "rsi": None, "atr": None,
        "stoch_k": None, "stoch_d": None
    }

    try:
        ticker_data = yf.Ticker(ticker_symbol)
     # 1. Fetch 5 years of daily bars for accurate EMA 200 mathematical
stabilization
        df = ticker_data.history(period="5y", interval="1d", prepost=True)

        if not df.empty and len(df) >= EMA_SLOW_PERIOD:
            df.columns = [col.lower() for col in df.columns]

    # 2. Extract the actual active extended hours price
        live_price = None
        try:
        # Fetch the live quote dictionary
            info = ticker_data.info

        # Ch eck if we are currently in post-market or pre-market and pull those
specific feeds
            post_market = info.get('postMarketPrice')
            pre_market = info.get('preMarketPrice')

            if post_market and post_market > 0:
                live_price = post_market
            elif pre_market and pre_market > 0:
                live_price = pre_market

        # Fallback: If .info fails or is empty, extract the last tick from a
1-minute live chart
            if not live_price:
                minute_df = ticker_data.history(period="1d", interval="1m",
prepost=True)
                if not minute_df.empty:
                    live_price = minute_df['Close'].iloc[-1]

        except Exception as e:
            print(f"Error fetching live extended price: {e}")
```

```
        # 3. Overwrite with the true active extended trading price if found
        if live_price and not pd.isna(live_price):
            print(f"True Active Extended Price Found: {live_price}")
            df.loc[df.index[-1], 'close'] = live_price
        else:
            print(f"Falling back to standard historical close:
{df['close'].iloc[-1]}")

        # 4. Indicators calculate on the dynamically modified final data point
            df['ema_50'] = ta.trend.ema_indicator(df['close'],
window=EMA_FAST_PERIOD)
            df['ema_200'] = ta.trend.ema_indicator(df['close'],
window=EMA_SLOW_PERIOD)

    ### based on Google Advice
            print(df[['close', 'ema_50', 'ema_200']].tail(1))

            ### based on Google Advice

            if df['ema_200'].isna().all():
                result_template["message"] = "EMA200 calculation generated
insufficient historical data."
                return result_template

        macd_obj = ta.trend.MACD(df['close'])
        df['macd'] = macd_obj.macd()
        df['macd_signal'] = macd_obj.macd_signal()

        df['adx'] = ta.trend.adx(df['high'], df['low'], df['close'],
window=ADX_PERIOD)
        df['rsi'] = ta.momentum.rsi(df['close'], window=RSI_PERIOD)
        df['atr'] = ta.volatility.average_true_range(df['high'], df['low'],
df['close'], window=ATR_PERIOD)

        df['stoch_k'] = ta.momentum.stoch(df['high'], df['low'], df['close'],
window=STOCH_K_PERIOD)
        df['stoch_d'] = ta.momentum.stoch_signal(df['high'], df['low'],
df['close'], window=STOCH_K_PERIOD, smooth_window=STOCH_D_PERIOD)

        latest = df.iloc[-1]
        prev   = df.iloc[-2]

        metrics = {
            "ticker": ticker_symbol, "price": round(latest['close'], 2),
            "ema_50": round(latest['ema_50'], 2), "ema_200":
round(latest['ema_200'], 2),
            "macd": round(latest['macd'], 3), "macd_signal":
round(latest['macd_signal'], 3),
            "adx": round(latest['adx'], 2), "rsi": round(latest['rsi'], 2),
```

```
            "atr": round(latest['atr'], 3), "stoch_k": round(latest['stoch_k'], 2),
            "stoch_d": round(latest['stoch_d'], 2)
        }

        price_distance_from_ema50 = metrics["price"] - metrics["ema_50"]

        # PHASE 1: Macro Trend Filter
        if not (metrics["price"] > metrics["ema_50"] > metrics["ema_200"]):
            metrics.update({"status": STATUS_IGNORE, "message": MSG_NO_TREND})
            return metrics

        # PHASE 2: Momentum Verification
        is_momentum_strong = (latest['macd'] > latest['macd_signal']) and
(latest['macd'] > 0) and (latest['adx'] > CFG_ADX_TREND_STRONG)

        # PHASE 3: Extension Check
        reasons_for_top = []
        if price_distance_from_ema50 > (CFG_ATR_EXT_MULT * metrics["atr"]):
            reasons_for_top.append(f"Price too far from EMA50 (Dist:
{price_distance_from_ema50:.2f} > Max: {CFG_ATR_EXT_MULT * metrics['atr']:.2f})")
        if latest['rsi'] > CFG_RSI_OVERBOUGHT:
            reasons_for_top.append(f"RSI Exhausted ({metrics['rsi']} >
{CFG_RSI_OVERBOUGHT})")
        if latest['adx'] > CFG_ADX_EXTREME:
            reasons_for_top.append(f"ADX Trend Hyper-Extended ({metrics['adx']} >
{CFG_ADX_EXTREME})")

        if reasons_for_top:
            combined_reason_msg = f"{MSG_EXTREME_TOP} Reasons: " + " |
".join(reasons_for_top)
            metrics.update({"status": STATUS_DO_NOT_BUY, "message":
combined_reason_msg})
            return metrics

        # PHASE 4: Timing / Triggers
        stoch_crossed_up = (prev['stoch_k'] <= prev['stoch_d']) and
(latest['stoch_k'] > latest['stoch_d'])
        stoch_is_oversold = latest['stoch_k'] < CFG_STOCH_OVERSOLD
        price_in_safe_buy_zone = price_distance_from_ema50 <= (CFG_ATR_SAFE_MULT *
metrics["atr"])

        if is_momentum_strong and price_in_safe_buy_zone and stoch_is_oversold and
stoch_crossed_up:
            metrics.update({"status": STATUS_BUY_SIGNAL, "message":
MSG_BUY_TRIGGER})
        elif is_momentum_strong and not price_in_safe_buy_zone:
            metrics.update({"status": STATUS_WATCHLIST, "message":
f"{MSG_WAIT_PULLBACK} (Dist from EMA50: {price_distance_from_ema50:.2f})"})
        else:
            metrics.update({"status": STATUS_HOLD, "message": MSG_HOLD_TREND})
```

```
        return metrics

    except Exception as e:
        result_template["message"] = f"Skipped (Internal Error: {str(e)})"
        return result_template

# ================================================================================
# 4. TICKER LOADER ENGINE (CSV FILE / PARSER)
# ================================================================================
def load_tickers_from_file(csv_path: str) -> list:
    if not csv_path or not os.path.isfile(csv_path):
        print(f"ⓘ[] Input file target '{csv_path}' empty or missing. Falling back
to default baseline watchlist...", flush=True)
        return DEFAULT_FALLBACK_WATCHLIST

    try:
        df_input = pd.read_csv(csv_path)
        found_col = [col for col in df_input.columns if col.strip().lower() in
["tickers", "ticker", "symbol"]]

        if not found_col:
            print(f"⚠ Error: File must contain a 'Ticker', 'Tickers' or 'Symbol'
header column. Reverting to system defaults.", flush=True)
            return DEFAULT_FALLBACK_WATCHLIST

        # FIXED: Explicitly extraction of index [0] parses column key as flat
series
        target_column_string = found_col[0]
        tickers =
df_input[target_column_string].dropna().astype(str).str.strip().str.upper().unique(
).tolist()
        return [t for t in tickers if t]
    except Exception as e:
        print(f"⚠ Error reading CSV file '{csv_path}': {str(e)}. Reverting to
system defaults.", flush=True)
        return DEFAULT_FALLBACK_WATCHLIST

# ================================================================================
# 5. MAIN EXECUTION BLOCK CLI RUNNER PIPELINE WITH SORTED OUTPUT
# ================================================================================

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Multi-Day Momentum Live Scanner
Engine v1.7")
    parser.add_argument("-c", "--codes", type=str, nargs="+", help="CLI 1: Direct
stock codes list")
    parser.add_argument("-i", "--input", type=str, default=DEFAULT_INPUT_CSV,
help="CLI 2: Path to input watchlist CSV file")
    parser.add_argument("-o", "--output", type=str, default=DEFAULT_OUTPUT_CSV,
```

```
help="CLI 3: Path to output log CSV file")
    args = parser.parse_args()
    source_message = ""
    if args.codes:
        raw_list = []
        for segment in args.codes:
            raw_list.extend(segment.split(','))
        watchlist = [t.strip().upper() for t in raw_list if t.strip()]
        source_message = f"Direct Command Line Input (-c) [{len(watchlist)} Tickers
Loaded]"
    else:
        watchlist = load_tickers_from_file(args.input)
        if set(watchlist) == set(DEFAULT_FALLBACK_WATCHLIST) and not
os.path.isfile(args.input):
            source_message = "Default System Watchlist Baseline (Fallback)"
        else:
            source_message = f"Input CSV File (-i) -> {args.input}
[{len(watchlist)} Tickers Loaded]"
    if not watchlist:
        print("❌ Error: Watchlist resolution produced 0 items. Exiting execution
loop.", flush=True)
        sys.exit(1)
    run_timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    total_tickers = len(watchlist)
    print("=" * 80, flush=True)
    print(f"🚀 STARTING LIVE MOMENTUM SCANNER | {run_timestamp}", flush=True)
    print(f"📥 Watchlist Source  : {source_message}", flush=True)
    print(f"💾 Historical Log Target: {args.output}", flush=True)
    print("=" * 80, flush=True)
    unordered_results = []
    counts = {STATUS_BUY_SIGNAL: 0, STATUS_WATCHLIST: 0, STATUS_HOLD: 0,
STATUS_DO_NOT_BUY: 0, STATUS_IGNORE: 0, STATUS_ERROR: 0}
    # Process Tickers with explicit counter indicators
    for idx, ticker in enumerate(watchlist, start=1):
        print(f"[{idx}/{total_tickers}] Running engine for {ticker}...",
flush=True)
        data_packet = evaluate_stock_momentum(ticker)
        data_packet["timestamp"] = run_timestamp
        unordered_results.append(data_packet)
        counts[data_packet["status"]] += 1
    # Group and sort the console outputs by priority order
    sorted_results = sorted(unordered_results, key=lambda x:
SORT_ORDER_PRECEDENCE.index(x["status"]) if x["status"] in SORT_ORDER_PRECEDENCE
else len(SORT_ORDER_PRECEDENCE))
    # --- TERMINAL SUMMARY OUTPUT ---
    print("\n" + "=" * 80, flush=True)
    print(f"📊 SUMMARY EXECUTION REPORT | {run_timestamp}", flush=True)
    print("=" * 80, flush=True)
    print(f"▶ Total Tickers Analyzed : {total_tickers}", flush=True)
    print(f"▶ BUY SIGNAL Count       : {counts[STATUS_BUY_SIGNAL]}", flush=True)
```

```
    print(f"▶ WATCHLIST Count        : {counts[STATUS_WATCHLIST]}", flush=True)
    print(f"▶ HOLD Count             : {counts[STATUS_HOLD]}", flush=True)
    print(f"▶ DO NOT BUY Count       : {counts[STATUS_DO_NOT_BUY]}", flush=True)
    print(f"▶ IGNORE Count           : {counts[STATUS_IGNORE]}", flush=True)
    print(f"▶ ERROR/SKIPPED Count    : {counts[STATUS_ERROR]}", flush=True)
    print("-" * 80, flush=True)
    print(f"{'CODE':<8} | {'STATUS':<12} | {'EXECUTION MESSAGE'}", flush=True)
    print("-" * 80, flush=True)
    for item in sorted_results:
        print(f"{item['ticker']:<8} | {item['status']:<12} | {item['message']}",
flush=True)
    print("=" * 80, flush=True)
    # --- CSV FILE GENERATION ---
    df_log = pd.DataFrame(unordered_results)
    columns_order = ["timestamp", "ticker", "status", "message", "price", "ema_50",
"ema_200", "macd", "macd_signal", "adx", "rsi", "atr", "stoch_k", "stoch_d"]
    df_log = df_log[columns_order]

    try:
        if os.path.dirname(args.output):os.makedirs(os.path.dirname(args.output),
exist_ok=True)
        file_exists = os.path.isfile(args.output)
        df_log.to_csv(args.output, mode='a', index=False, header=not file_exists)
        print(f"💾 Historical analysis report updated successfully at:\n👉
{args.output}\n", flush=True)
    except Exception as e:
        print(f"❌ Failed to write log data to disk: {e}", flush=True)
```